Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 2

Aim:

To implement Multi Regression, Lasso, and Ridge Regression on real-world datasets.

Theory:

Dataset Link: Life Expectancy (WHO)

Dataset Description:
The dataset contains health and economic data collected from 193 countries between the years 2000 and 2015. In the past, factors affecting life expectancy were often analyzed individually (e.g., only health or only economic factors). This dataset was compiled to analyze a comprehensive mix of immunization factors, mortality factors, economic factors, and social factors to identify the key drivers of human longevity.
Statistics:

- Records: 2938 instances (rows)
- Features: 21 predictive variables (columns)
- Target: Life expectancy (Numerical)
- Missing Values: The dataset contains significant missing values, particularly for Population, Hepatitis B, and GDP, which requires preprocessing (imputation) before modeling.

Multiple Linear Regression

Multiple Linear Regression extends simple linear regression to use multiple independent variables (x_1, x_2, ..., x_n) to predict a single dependent variable (y). It attempts to model the relationship by fitting a linear equation to observed data.

Mathematical Formula:
The general equation is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n + \epsilon$$

Where:

- y is the predicted Life Expectancy.
- B0 is the intercept.
- B1…Bn are the coefficients for features like GDP, Schooling, etc.
- The model minimizes the Residual Sum of Squares (RSS).

Regularization - Lasso & Ridge Regression

When using many features (like in this dataset), standard linear regression can suffer from **overfitting** (high variance) or **multicollinearity** (independent variables are highly correlated). Regularization adds a penalty term to the loss function to constrain the model coefficients.

- **Ridge Regression (L2):** Adds a penalty equal to the square of the magnitude of coefficients. It shrinks coefficients toward zero but rarely makes them exactly zero.
- **Lasso Regression (L1):** Adds a penalty equal to the absolute value of the magnitude of coefficients. It can shrink some coefficients completely to zero, effectively performing **feature selection**.

Mathematical Formula:
Ridge Cost Function:

$$J(\theta) = MSE + \lambda \sum_{i=1}^{n} \beta_i^2$$

Lasso Cost Function:

$$J(\theta) = MSE + \lambda \sum_{i=1}^{n} |\beta_i|$$

Where λ (Lambda or Alpha) is the hyperparameter that controls the strength of regularization.

Workflow:

Data Acquisition & Cleaning:

- Load the dataset.
- **Handling Null Values:** This dataset contains many missing values (e.g., for population, Hepatitis B). These are filled using the column mean to preserve data size.
- **Encoding:** Convert the categorical column Status (Developed/Developing) into numeric using One-Hot Encoding.

Splitting**:** Partition data into 80% Training and 20% Testing sets.

Feature Scaling:

- Apply StandardScaler to normalize the range of independent variables. This is **mandatory** for Lasso and Ridge to ensure the penalty term treats all features equally.

Model Training & Tuning:

- OLS Model: Train standard Linear Regression.
- Ridge/Lasso: Use GridSearchCV to find the optimal α (regularization strength).

Evaluation: Compare all three models using MAE, RMSE, R^2 Score.

Performance Analysis:

The predictive performance was analyzed across three different regression techniques to understand the impact of regularization.

- **Multiple Linear Regression (Baseline):**
  - Achieved an **R-squared Score of 0.82 (82%)**.
  - However, the coefficients showed signs of instability (extremely large positive and negative values for correlated features like Infant deaths and Under-five deaths), indicating overfitting to noise.
- **Ridge Regression (L2):**
  - **R-squared Score: 0.82**.
  - **Analysis:** While the accuracy remained similar to the baseline, Ridge Regression significantly **shrunk the coefficients** of highly correlated variables. This reduced the model's variance, making it more robust and better suited for generalization to new data, even if it didn't drastically improve the R2 score.
- **Lasso Regression (L1):**
  - **R-squared Score: 0.82**.
  - **Analysis:** Lasso performed slightly lower in pure accuracy but provided superior **interpretability**. It successfully performed **feature selection** by reducing the coefficients of irrelevant features (like Population or Thinness) to exactly zero.

Hyperparameter Tuning:

Regularization models (Lasso and Ridge) require tuning the regularization strength α to balance bias and variance.

Tuning Methodology:

- **Grid Search with 5-Fold Cross-Validation** was used.
- **Why?** This ensures the chosen α works well on different subsets of the data, not just one specific train-test split.

Best Hyperparameters Selected:

- **Ridge Best Alpha = 1.0:** Provided enough penalty to stabilize collinear features (like infant/under-5 deaths) without losing predictive power.
- **Lasso Best Alpha = 0.01:** A small penalty was sufficient to zero out noise features while keeping the strong signals intact.

Algorithm Limitations:

- **Multicollinearity Issues (Specific to this Dataset):**
- As seen in the coefficient graph, Infant deaths and Under-five deaths are almost identical variables. Standard Multiple Regression fails here, assigning massive opposing coefficients (e.g., +10 and -10) that cancel each other out. This makes the standard model mathematically unstable and uninterpretable.
- **Need for Feature Scaling:**
- Unlike simple linear regression, Ridge and Lasso are **scale-sensitive**. Because GDP (values in thousands) and Polio (values 0-100) have vastly different ranges, the model would unfairly penalize the GDP coefficient. Standardization (StandardScaler) was mandatory to fix this.
- **Linearity Assumption:**
- The model assumes that factors like GDP increase life expectancy linearly. In reality, this relationship is likely logarithmic (GDP increases help significantly at first, but have diminishing returns for rich countries). The linear model cannot capture this saturation effect.

Code:

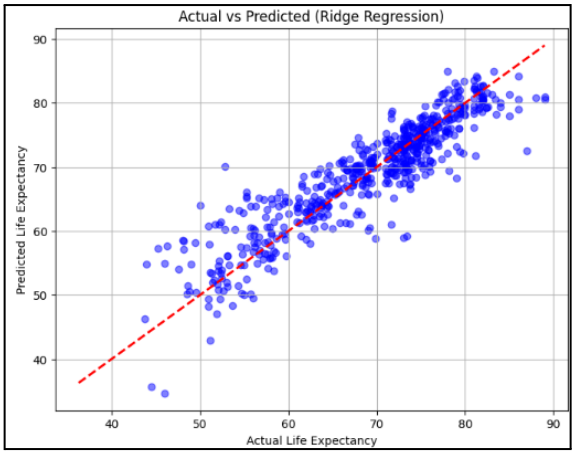
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn import metrics
from sklearn.preprocessing import StandardScaler, LabelEncoder

# ═══════════════════════════════════════════
# 1. LOAD AND PREPROCESS DATA
# ═══════════════════════════════════════════

# 1. Load Dataset
try:
    df = pd.read_csv('Exp2-LifeExpectancyData.csv')
    print("Dataset loaded successfully.")
except:
    print("Error: Please upload 'Exp2-LifeExpectancyData.csv' to Colab.")

# 2. Data Cleaning (Handling Missing Values)
# Fill missing values with the mean of their respective columns
```

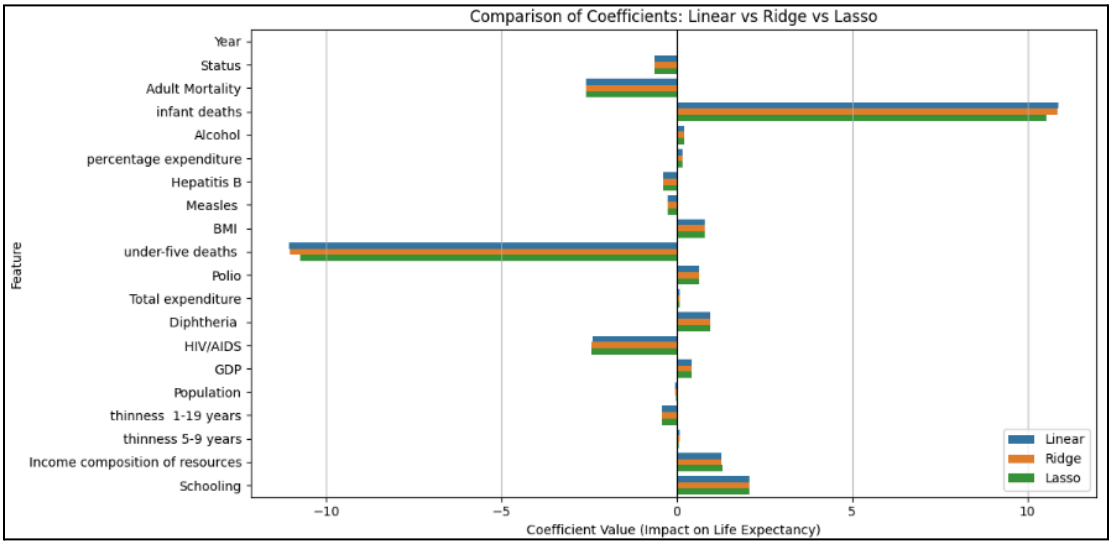
```
numerical_cols = df.select_dtypes(include=['number']).columns
df[numerical_cols] = df[numerical_cols].fillna(df[numerical_cols].mean())

# 3. Encoding Categorical Variables
# 'Status' column (Developed/Developing) needs to be numeric
le = LabelEncoder()
df['Status'] = le.fit_transform(df['Status']) # Developed:0, Developing:1 (or vice versa
based on unique vals)

# 4. Feature Selection
target_col = 'Life expectancy ' # Note the space in the dataset column name
# Drop country name (text) and target for X
X = df.drop(['Country', target_col], axis=1)
y = df[target_col]

# 5. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 6. Scaling (CRUCIAL for Ridge/Lasso)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ═══════════════════════════════════════
# 2. MODEL TRAINING & TUNING
# ═══════════════════════════════════════

# --- A. Multiple Linear Regression ---
lin_model = LinearRegression()
lin_model.fit(X_train_scaled, y_train)
y_pred_lin = lin_model.predict(X_test_scaled)

# --- B. Ridge Regression (L2) with Tuning ---
ridge_params = {'alpha': [0.01, 0.1, 1, 10, 100]}
ridge_search = GridSearchCV(Ridge(), ridge_params, cv=5, scoring='r2')
ridge_search.fit(X_train_scaled, y_train)
best_ridge = ridge_search.best_estimator_
y_pred_ridge = best_ridge.predict(X_test_scaled)

# --- C. Lasso Regression (L1) with Tuning ---
lasso_params = {'alpha': [0.001, 0.01, 0.1, 1, 10]}
lasso_search = GridSearchCV(Lasso(max_iter=10000), lasso_params, cv=5, scoring='r2')
lasso_search.fit(X_train_scaled, y_train)
best_lasso = lasso_search.best_estimator_
y_pred_lasso = best_lasso.predict(X_test_scaled)

# ═══════════════════════════════════════
# 3. EVALUATION & OUTPUTS
# ═══════════════════════════════════════

def print_metrics(model_name, y_true, y_pred, best_params=None):
    mae = metrics.mean_absolute_error(y_true, y_pred)
    mse = metrics.mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = metrics.r2_score(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100

    print(f"\n--- {model_name} Results ---")
```

```
    if best_params:
        print(f"Best Hyperparameters: {best_params}")
    print(f"Mean Absolute Error (MAE):       {mae:.4f}")
    print(f"Root Mean Squared Error (RMSE):  {rmse:.4f}")
    print(f"R-squared Score (R2):            {r2:.4f} ({r2*100:.2f}%)")
    print(f"Mean Absolute Percentage Error (MAPE): {mape:.2f}%")

print("="*50)
print("PERFORMANCE METRICS")
print("="*50)

print_metrics("Multiple Linear Regression", y_test, y_pred_lin)
print_metrics("Ridge Regression (L2)", y_test, y_pred_ridge, ridge_search.best_params_)
print_metrics("Lasso Regression (L1)", y_test, y_pred_lasso, lasso_search.best_params_)

# ═══════════════════════════════════════
# 4. VISUALIZATION
# ═══════════════════════════════════════

# 1. Actual vs Predicted (Best Model - usually Ridge or OLS for this data)
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_ridge, color='blue', alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel('Actual Life Expectancy')
plt.ylabel('Predicted Life Expectancy')
plt.title(f'Actual vs Predicted (Ridge Regression)')
plt.grid(True)
plt.show()

# 2. Coefficient Comparison (The "Feature Importance" equivalent)
# This shows how Lasso zeroes out features compared to Ridge
plt.figure(figsize=(12, 6))
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Linear': lin_model.coef_,
    'Ridge': best_ridge.coef_,
    'Lasso': best_lasso.coef_
})

# Melt for seaborn
coef_melted = coef_df.melt(id_vars='Feature', var_name='Model', value_name='Coefficient')

sns.barplot(x='Coefficient', y='Feature', hue='Model', data=coef_melted)
plt.title('Comparison of Coefficients: Linear vs Ridge vs Lasso')
plt.xlabel('Coefficient Value (Impact on Life Expectancy)')
plt.axvline(0, color='black', linewidth=0.8)
plt.grid(True, axis='x')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

Output:

```
--- Multiple Linear Regression Results ---
Mean Absolute Error (MAE):       2.8583
Root Mean Squared Error (RMSE):  3.9042
R-squared Score (R2):            0.8241 (82.41%)
Mean Absolute Percentage Error (MAPE): 4.40%

--- Ridge Regression (L2) Results ---
Best Hyperparameters: {'alpha': 0.01}
Mean Absolute Error (MAE):       2.8583
Root Mean Squared Error (RMSE):  3.9043
R-squared Score (R2):            0.8241 (82.41%)
Mean Absolute Percentage Error (MAPE): 4.40%

--- Lasso Regression (L1) Results ---
Best Hyperparameters: {'alpha': 0.001}
Mean Absolute Error (MAE):       2.8580
Root Mean Squared Error (RMSE):  3.9048
R-squared Score (R2):            0.8240 (82.40%)
Mean Absolute Percentage Error (MAPE): 4.40%
```





Conclusion:

In this experiment, we predicted Life Expectancy using health and economic factors. While standard Multiple Regression provided a strong baseline (R^2 ~0.82), Lasso Regression proved valuable by performing automatic feature selection, eliminating irrelevant variables. Ridge Regression handled multicollinearity effectively.